14
Design A Search Autocomplete System

Most popular products and services have some form of autocomplete functionality. For example, when you type in the search box, we are presented a list. The list is refined as we continuously type more and more characters. In this chapter, we design such a system. Typeahead suggestion is a feature that suggests a list of words to users based on the characters they have typed. For example, when we type in "google" into the search engine, we may see the following suggestions.

Figure 1

Step 1 - Understand the problem and establish design scope

For this topic, we may have many different definitions of what it is an autocomplete system. In some organizations, it is a complicated machine learning system.

Interviewer: Is it a spelling check system?

Candidate: No, correcting the misspelled word is beyond the scope of this system.

Interviewer: How many suggestions should we show?

Candidate: We need to specify a number in our system. We will discuss this later.

Interviewer: How do we rank the suggestions?

Candidate: The suggestions are ranked based on popularity.

Interviewer: Do we need to support multi-word queries?

Candidate: No, only single-word queries are supported.

Interviewer: Do we need to support a English?

Candidate: Yes, if time allows at the end, we will discuss multi-language support.

Interviewer: How about capitalization and special characters?

Candidate: We can assume all queries are lowercase alphabetical characters.

Interviewer: Good, we can focus on the core algorithm.

Candidate: Sounds good.

Requirements

Some crucial requirements are:

- Fast response time: no user wants a search page, autocomplete suggestions must show up fast enough. An article about Facebook's typeahead suggestion suggests that the system should be able to return results within a few milliseconds.
- Relevant typeahead suggestions: should be relevant to the search term.
- Sorted: Results returned by the system must be sorted by popularity or other ranking criteria.
- Scalable: The system can handle a high traffic volume.
- Highly available: The system should still work and continue when part of the system is offline. Slow, stale or unavailable search suggestions are better than no suggestions at all.

Back of the envelope estimation

- 10 million DAU (daily active users) visit Google.
- On average each person is expected to issue 10 searches per day.
- Each query has 20 characters.
- Each character is stored in ASCII encoding, 1 character = 1 byte.
- Assume a query contains 4 words and each word contains 5 characters on average.
- Then, it is 20 bytes per query.
- For every character entered into the search box, a request is sent to the backend for autocomplete suggestions. On average, 20 requests are sent for each search query. For example, the following 6 requests are sent to the backend for the search query "google".

```
google
google
google
google
google
google
```

- Assume 2 billion queries are issued per day.

Based on the following data, we can estimate the peak QPS and storage requirements. Peak QPS is the maximum query per second (QPS) that can be processed by a service in a given time period. We can assume the peak QPS is two times of the average QPS.

- Average QPS = 2 billion queries / 24 hours / 3600 seconds = 24,000 QPS
- Peak QPS = 24000 * 2 = 48,000
- Assume 20% of the queries contain new terms. For example, 20 billion queries per day, 20% of 20 billion queries = 4 billion new queries per day.
- Storage = 4 billion * 20 bytes = 80 GB

Step 2 - Propose high-level design and get buy-in

At the high level, the system is divided into two parts:

- Data gathering service: gathers user input queries and aggregates them in real-time. Real-time processing is not practical for large data sets and it also affects the system's response time. We will discuss how to gather data in a scalable way.
- Query service: given a search query or prefix, return 5 most frequently used terms.

Data gathering service

Let's use a simplified example to see how data gathering service works. Assume we have a frequency table that stores the query frequy table. Initially, the frequency table is empty. We assume users have typed the following queries: twitch, twitch, twitter, twitter, twitch, twitch, twitter, twitter, twitter, twitch. Figure 2 shows the frequency table.

query varchar	day date	hour int	frequency int
twitch	2018-01-01	0	2
twitter	2018-01-01	0	3
twitch	2018-01-01	1	4
twitter	2018-01-01	1	3

Figure 2

Query service

Assume we have the following table to store 5 terms for each prefix.

- Query: a string to store query string.
- Frequency: an integer to store the number of times a query has been searched.

query varchar	frequency
twitch	35
twitter	29
twitch app	26
twitch apps	22
twitch apps for pc	19

Figure 3

To get top 5 frequently used queries, we use the following query:

```
SELECT query, frequency
FROM aggr_table
WHERE day = '2018-01-01'
AND prefix = 'twitch'
ORDER BY frequency DESC
LIMIT 5
```

Figure 4

When a user types "tw", "twi" and "twitc", the following four prefixes are displayed (Figure 5), assuming the frequency table is stored in cache.

```
tw
twi
twit
twitc
```

Figure 5

To get top 5 frequently used queries, we use the following query:

```
SELECT query, frequency
FROM aggr_table
WHERE day = '2018-01-01'
AND prefix = 'twitc'
ORDER BY frequency DESC
LIMIT 5
```

Figure 6

When the data volume is low, it works well. But when it is very large, it becomes slow. We need to scale the database or cache. Assume we have the following frequency table:

query	frequency
twitch	35
twitter	29
twitch app	26
twitch apps	22
twitch apps for pc	19

Figure 7

Since the data volume is very large, we can't fit it into a single server. We need to shard the frequency table. Assume we use the following frequency table:

query	frequency
twitch	35
twitter	29
twitch app	26
twitch apps	22
twitch apps for pc	19

Figure 8

Shard the data based on the first character. For example, sharding frequency table with the first character in ASCII. Assume we have the following frequency table:

query	frequency
twitch	35
twitter	29
twitch app	26
twitch apps	22
twitch apps for pc	19

Figure 9

Sharding based on the first character can still cause hotspots. For example, there are many English words that start with the letter "t". For better scalability and avoid hotspots, a trie (prefix tree) data structure is a scalable and efficient way to build the query service. Let's explore how it works.

Trie data structure

Trie (pronounced "try") is a tree-like data structure that compactly stores strings. They are ideal for prefix lookup. Trie data structure is a good candidate for finding all valid strings that share a common prefix. In our case, the strings are search queries.

- Trie has a root node that represents an empty string.
- Each node stores 26 children (a-z), or 36 children (a-z and 0-9) if we need to include numbers.
- Each node represents a single character.
- Given a node, we can traverse up to 26 children to find the next character in the query.
- Each node represents a string (prefix) composed of characters from the root node to this node.

Advantages of using trie data structure to store and search for prefixes are:

- Fast response time: the worst case time complexity for looking up a prefix is O(p), where p is the length of the prefix. For example, the time to find the prefix "tw" is proportional to the length of the prefix, which is 2.
- Fast operations in sorted string.
- Trie can be stored in memory.
- Trie can be serialized and deserialized into/from disk.

Let's look at how to build a trie. Assume we have the following 6 queries and each query has a frequency:

- tree: 10
- true: 35
- try: 29
- toy: 14
- wish: 25
- winter: 40

Figure 10 shows the trie data structure with the queries and their frequencies. Each node stores a character and 26 children. For simplicity, we only show 4 children for each node.

Figure 10

Step 1: Find the children of the prefix. For example, the children of the prefix "tr" are "e" and "u".

Step 2: Find the children of each of the two prefixes in step 1. In our example, there are four prefixes: tre, tru, wre, wru.

Step 3: Find the children of each of the four prefixes in step 2. In our example, there are two prefixes: true and wrue.

Step 4: Find the children of each of the two prefixes in step 3. In our example, there are two prefixes: true and wrue.

Step 5: Traverse the sub-tree to get all valid children. A child is a valid term if it is stored in a leaf node. In our example, the following terms are found: tree, true, try, trie, wrie, wrue.

Step 6: Sort the children and get top 5. In our example, the following terms are found: true, tree, try, wrie, wrue.

This algorithm is inefficient. In the worst case, we need to traverse the entire trie to get 5 children. We can improve the query service by limiting the maximum length of a prefix to 50 characters. We also need to cache the top 5 search queries at each node. Let's see how it works.

- Limit the length of a prefix: We assume the longest prefix we will use to match a query is 50 characters. This limits the number of times we need to perform step 1 to 4 mentioned above.
- Cache top 5 search queries at each node: Storing the top 5 search queries at each node speeds up the retrieval process. It is sufficient for all but the most extreme prefixes.

Figure 11 shows the updated trie data structure with the queries and their frequencies.

Figure 11

Let's see how the time complexity of the algorithm after applying these two optimizations.

- Find the prefix node: Time complexity: O(1)
- Return top 5 from the cached search queries: Time complexity: O(1)

So the time complexity of the algorithm is O(1). It is a significant improvement from the original O(p).

Data gathering service

In previous design, whenever a user types a search query, data is updated in real-time. This approach is not practical for the following two reasons:

- Users may keep refining their search query. For example, the first query may be twitch, then twitch app, then twitch app for pc.
- The aggregation (top 5 search queries) may change a lot. Thus, it is not practical to update the frequency table.

In a large scale data gathering service, we assume billions data events (queries) are streamed into the system. Kafka is a popular choice to store streaming data. It is fast, scalable, and fault-tolerant. Figure 12 shows the data gathering service with Kafka.

Figure 12

Analytics logs: A server log data about user's actions. For example, a server log can contain a record that contains the following data:

field	description
uid	user id
query	search query
lang	language (English)
country	country (US)
agent	client agent (web, mobile, etc)
time	time stamp (2018-01-01 00:00:00)

Table 1

Aggregations: The size of analytics logs is usually very large, and data is in the raw form. We need to aggregate data so it can be easily consumed by various jobs.

Aggregation can be based on many factors. For example, the most useful aggregation could be based on the last query, aggregation could be based on trending (most frequent) or based on user's location. On the other hand, aggregating data frequently, say on a per minute basis or per second basis, is a good choice for real-time search suggestions. It is also CPU and memory intensive. We will use the following aggregation:

Aggregation window: We can aggregate data on a per request basis or every 1 minute, every 1 hour, or every 1 day. For simplicity, we choose to aggregate data on a weekly basis.

Table 2 shows an example of aggregated weekly data. The "time" field represents the start date of a week. Frequency" field is the sum of the occurrences for the corresponding query in that week.

query	time	frequency
tree	2019-10-22	10
tree	2019-10-29	7,000
tree	2019-11-12	500
true	2019-10-22	35
true	2019-10-29	800
true	2019-11-12	70
try	2019-10-22	29
try	2019-10-29	1,000
try	2019-11-12	90

Table 2

Workers: Workers are a set of servers that perform asynchronous and scheduled jobs. They build the trie data structure and persist it to the trie database.

Trie cache: Trie cache is a distributed cache system that keeps trie in memory for fast read. It takes weekly snapshots of the DB.

Trie DB: Trie DB is the persistent storage. It can be implemented as a NoSQL database or key-value store. There are many options to choose from to store a hierarchical data structure. In our design, we choose document DB because it is easy to serialize a new trie and it is also easy to add new data to the trie.

Figure 13 shows the updated design after we introduce the analytics log, aggregation, workers, trie cache, and trie DB.

Figure 13

- Search queries are logged to the analytics log.
- The data is aggregated weekly.
- Workers are used to build the trie data structure based on the aggregation results.
- Trie data is cached in the trie cache.
- Trie data is persisted to the trie database.

Our design is mainly data gathering service and query service. We will explore how to scale the data gathering service and query service.

Step 3 - Design deep dive

In the high level design, we discussed the following components:

- Trie data gathering service
- Trie cache
- Trie database
- Query service
- Scale the storage
- Trie operations

The data structure

Hashtable is used in the query service to store the top 5 frequently used queries. The key-value pair is stored in the hashtable. For example, the following entry means that the query prefix "tw" has the following 5 queries: "twitch", "twitter", "twitch app", "twitch apps", "twitch apps for pc".

Hashtable is a fast data structure for looking up a key-value pair. It is very efficient for our use case. It works well if we know the exact prefix we want to look up. However, it does not support operations such as:

- Find the prefix "tw" and all its children
- Find the top 5 most frequently used queries
- Sort the results

Trie (prefix tree) is a good data structure to handle these operations. It is a tree-like data structure. Let's see how it works. Assume we have the following queries: tree, true, try, toy, wish, winter. Figure 14 shows the trie data structure with these queries.

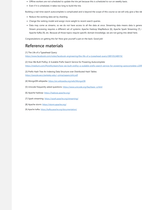

Figure 14

Step 1: Find the children of the prefix. For example, the children of the prefix "tr" are "e" and "u".

Step 2: Find the children of each of the two prefixes in step 1. In our example, there are four prefixes: tre, tru, wre, wru.

Step 3: Find the children of each of the four prefixes in step 2. In our example, there are two prefixes: true and wrue.

Step 4: Find the children of each of the two prefixes in step 3. In our example, there are two prefixes: true and wrue.

Step 5: Traverse the sub-tree to get all valid children. A child is a valid term if it is stored in a leaf node. In our example, the following terms are found: tree, true, try, trie, wrie, wrue.

Step 6: Sort the children and get top 5. In our example, the following terms are found: true, tree, try, wrie, wrue.

This algorithm is inefficient. In the worst case, we need to traverse the entire trie to get 5 children. We can improve the query service by limiting the maximum length of a prefix to 50 characters. We also need to cache the top 5 search queries at each node. Let's see how it works.

- Limit the length of a prefix: We assume the longest prefix we will use to match a query is 50 characters. This limits the number of times we need to perform step 1 to 4 mentioned above.
- Cache top 5 search queries at each node: Storing the top 5 search queries at each node speeds up the retrieval process. It is sufficient for all but the most extreme prefixes.

Figure 15 shows the updated trie data structure with the queries and their frequencies.

Figure 15

Let's see how the time complexity of the algorithm after applying these two optimizations.

- Find the prefix node: Time complexity: O(1)
- Return top 5 from the cached search queries: Time complexity: O(1)

So the time complexity of the algorithm is O(1). It is a significant improvement from the original O(p).

Data gathering service

In previous design, whenever a user types a search query, data is updated in real-time. This approach is not practical for the following two reasons:

- Users may keep refining their search query. For example, the first query may be twitch, then twitch app, then twitch app for pc.
- The aggregation (top 5 search queries) may change a lot. Thus, it is not practical to update the frequency table.

In large scale data gathering service, we assume billions data events (queries) are streamed into the system. Kafka is a popular choice to store streaming data. It is fast, scalable, and fault-tolerant. Figure 16 shows the data gathering service with Kafka.

Figure 16

- Search queries are logged to the analytics log.
- The data is aggregated weekly.

Our design is mainly data gathering service and query service. We will explore how to scale the data gathering service and query service.

Step 4 - Wrap up

Well, you finish the deep dive section. Let's recap what we have discussed.

- Trie data structure is used to store the search queries and their frequencies.
- Trie is an efficient data structure to handle prefix lookup.
- Limit the length of a prefix to 50 characters.
- Cache top 5 search queries at each node to speed up the retrieval process.
- Data gathering service uses Kafka to process user input queries in a near-real time fashion.
- Analytics logs are aggregated weekly.
- Aggregation window can be based on many factors such as last query, trending (most frequent), or based on user's location.
- Workers are used to periodically update the trie jar persistent the trie data structure to the trie database.
- Trie cache runs a cluster of servers to handle all read requests.

Data gathering service and query service can be scaled independently.

- Data gathering service: we can add more servers to handle more incoming data.
- Query service: we can add more servers to handle more search queries.

Scaling the storage is important for both data gathering service and query service. For simplicity, we only discuss how to scale the trie database.

- Shard a document database if the size of a single node exceeds limits. For example, MongoDB uses sharding to store growing data sets.
- Primary-secondary replication for high availability and data redundancy.

Congratulations on getting this far! Now give yourself pat on the back. Good job!

Reference materials

[1] Trie on Wikipedia.
[2] Kafka quickstart.
[3] Big Data Analytics in Asynchronous Parallel Computing https://ieeexplore.ieee.org/document/6676911/
[4] Scalable Trie-based Indexes for Efficient Prefix Matching in Key-value Stores https://www.cs.uwaterloo.ca/~tozsu/publications/thesis/MMarcus-MSc.pdf
[5] Prefix Hash Tree: An Indexing Data Structure over Distributed Hash Tables https://www.cs.berkeley.edu/~sylvia/papers/phtree.pdf
[6] Google autocomplete. https://support.google.com/websearch/answer/7368877?hl=en
[7] Google frequently asked questions https://www.google.com/intl/en_us/faq.html
[8] Internet Trends. https://www.kpcb.com/internet-trends
[9] Zipfian distribution. https://en.wikipedia.org/wiki/Zipf%27s_law
[10] Amazon DynamoDB. https://aws.amazon.com/dynamodb/